

Proctor — the agent that QAs other agents

UiPath AgentHack 2026 · Track 3: UiPath Test Cloud

Technical brief · Dan Mercede · github.com/OrionArchitekton/proctor (MIT) · Built with Claude Code, the coding agent

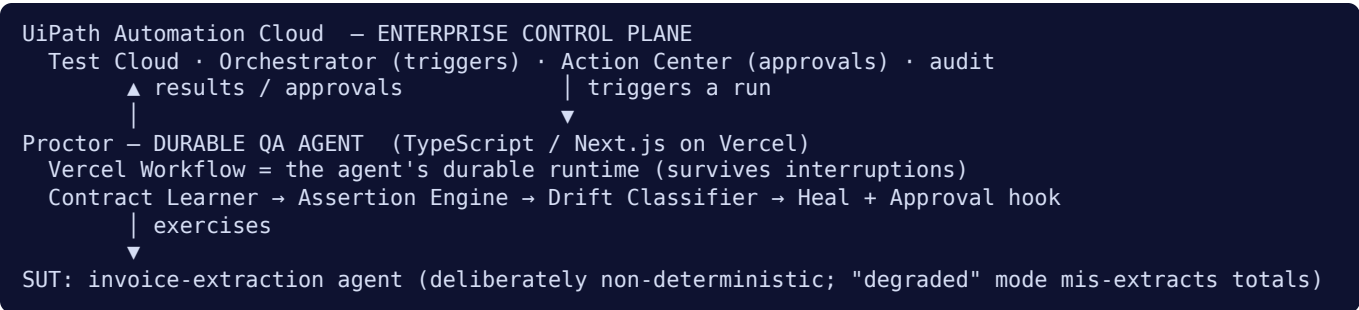
Proctor regression-tests non-deterministic AI automations — it learns each automation's behavioral contract, catches when a model or prompt change silently breaks it, classifies the drift, and escalates real regressions to a human, all governed by UiPath.

The problem & the method

AI automations are non-deterministic: the same input rarely yields byte-identical output, so naive `assertEqual(output, expected)` either flakes constantly or passes on garbage. Proctor solves this with a **behavioral contract + drift classification** approach:

- **Learn** — run the automation over sample inputs; synthesize a versioned `Contract` : per-field assertion *altitudes* (structural / exact / semantic) plus absolute domain **invariants** (e.g. `sum_line_items_eq_total`).
- **Run** — re-evaluate against the contract using invariants + semantic similarity, never brittle equality.
- **Classify** — a drift classifier decides **real-regression** (alert), **legitimate-evolution** (still correct → propose a contract patch), or **flaky** (within tolerance). This is the novel core: distinguishing "it broke" from "it changed but is still correct."
- **Escalate** — real regressions pause a **durable workflow on a human-approval hook** that survives interruptions (verified against the runtime event log, not mocked).

Architecture — two altitudes that never compete



One `UiPathGateway` adapter is the seam: `LocalGateway` (disk + dashboard, runs anywhere) ↔ `TestCloudGateway` (real UiPath REST). UiPath governs at the org level; Vercel Workflow makes the agent itself durable — they operate at different altitudes, so "UiPath orchestrates" and "the agent is durable" are both true.

Live UiPath verification — exercised against a real Labs tenant

Tenant `staging.uipath.com/hackathon26_529/DefaultTenant` · all four gateway surfaces hit with the actual code · reproduce: `doppler run -- env PROCTOR_GATEWAY=testcloud npx tsx scripts/live-gateway-probe.ts`

Surface	UiPath endpoint	Live evidence
Action Center (approval task)	tasks/GenericTasks/CreateTask (ExternalTask)	✓ created tasks 100000126–128
Orchestrator (triggered run)	odata/Jobs/...StartJobs	✓ job 67149929, State = Successful

Results channel (TestReport)	odata/Queues/...AddQueueItem	✓ Proctor_TestResults queue + items
Governance	– (no public audit-write API)	✓ honest warn-by-design

Note: Test Cloud's *Test Set execution* path is also wired (`StartTestSetExecution`); this Labs tenant gates Test-Case authoring behind a local UiPath Robot install, so the Orchestrator queue is the live results channel. Nothing is overclaimed — every still-unconfirmed field carries a `// VERIFY:` note in `testcloud.ts`.

Build & delivery evidence

- **76 unit tests + 1 pause/resume integration test** (genuine workflow suspend → resume), typecheck clean, Next.js build green.
- **Keyless & reproducible** for judges: `pnpm install && pnpm demo` (no API key, no UiPath creds) narrates the full logic; `pnpm dev` runs the live dashboard.
- Pure, dependency-injected engine (assertions / invariants / runner / learner / classifier / risk) — the testable core, isolated from all I/O.

Built with coding agents (bonus)

The Proctor solution — monorepo, engine, durable workflows, Next.js dashboard, test suite, and docs — was built by **Claude Code** (Anthropic's coding agent) through a spec → plan → test-driven-implementation pipeline. The git commit history is the evidence — a self-attestation, supported by a coherent feature-by-feature commit structure. (The live UiPath wiring against the Labs tenant was done by the operator.) This satisfies the "UiPath for Coding Agents" bonus under Platform Usage.

Repo: github.com/OrionArchitekton/proctor (MIT) · Built with: UiPath (Action Center · Orchestrator · Test Cloud) · Vercel Workflow · Anthropic Claude · Claude Code · TypeScript / Next.js